Week 1 · What is an information system?

Programming for Information Systems

People, data, rules — and your first lines of Python

Why learn programming when AI can write code?

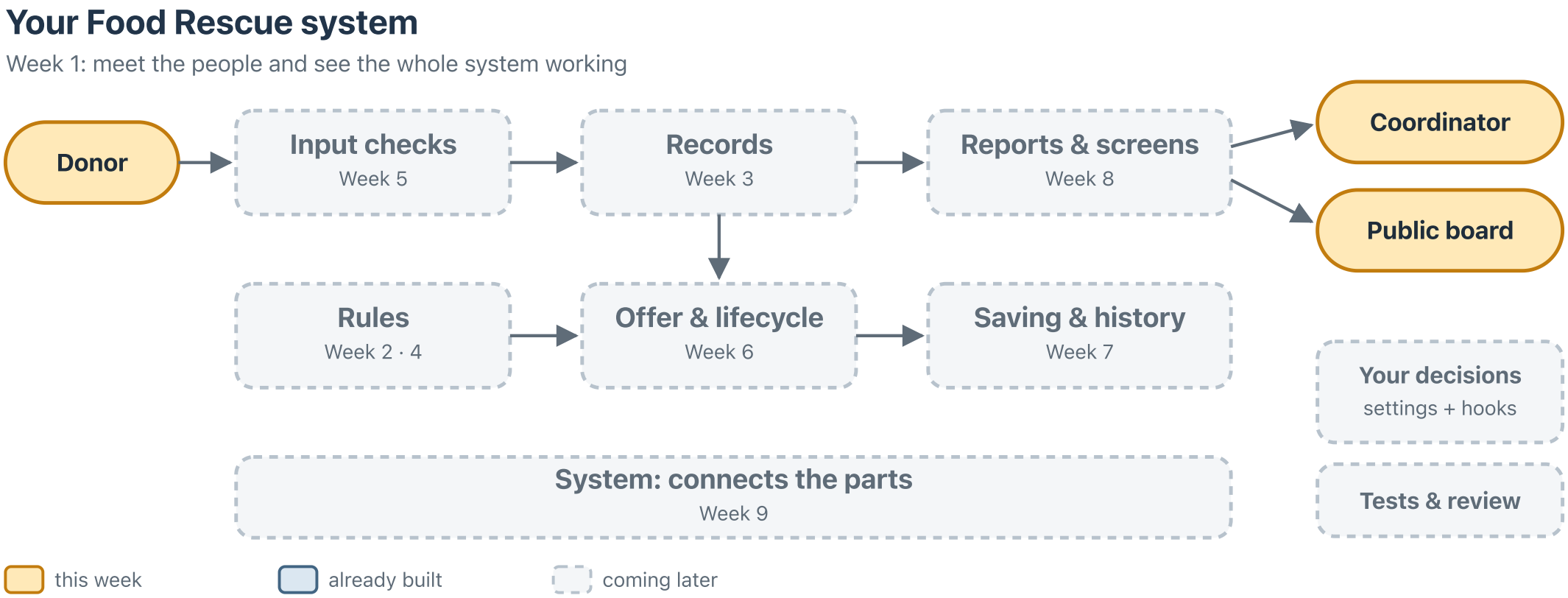
- AI can write a lot of code quickly.
- It cannot decide **what your system should do** for real people.
- It often writes code that runs but is **risky**: unclear, unchecked, unfair.

> In this course you learn enough Python to **read, test and judge** code — and you build a small system with your own decisions.

This week

Question	What is an information system, and who depends on it?
Python	values, names, types, arithmetic, text, f-strings
Principle 1	Clear names and units
You build	the one-line offer summary

Your system, by Week 10



Start with the destination

```
system = fr.System()
system.run_day()
print(system.report("coordinator"))
print(system.report("public"))
```

- 9 offers arrive; 5 are **refused at the door**.
- A coordinator **approves** or **rejects**; volunteers **pick up** and **deliver**.
- Two people see two different views.

An information system is more than code

An information system = people + data + rules + process + output

Campus Food Rescue as an example

People

donors, coordinators, volunteers, partners

Data

offers: food, portions, hours left, status

Rules

approve first; 20+ portions needs a team

Technology

Python, files, a web page: the tools, not the system

Process

offer → approve → pick up → deliver

Output

a coordinator list, a public board

Python foundations

Values, names and text

Values have types

```
print(type(30))            # int    — whole number
print(type(2.5))           # float  — decimal number
print(type("Bread rolls")) # str    — text
print(type(True))          # bool   — True or False
```

- Text always has quotes: `"30"` is text, `30` is a number.
- The type decides what you can do with a value.

A name points to a value

```
portions = 30
portions = portions + 12
print(portions)     # 42
```

- `=` means "let this name point to this value".
- It is **not** the = of mathematics: `portions = portions + 12` makes sense.
- A good name says the **meaning** and the **unit**.

Arithmetic: which answer is useful?

```
portions = 30
volunteers = 4
print(portions / volunteers)   # 7.5
print(portions // volunteers)  # 7   whole portions each
print(portions % volunteers)   # 2   left over
```

> You cannot carry half a portion. The **correct** calculation depends on what the number is for.

Text and numbers, f-strings

```
print(3 + 4)          # 7
print("3" + "4")      # 34  (joins text)
print("Portions: " + str(30))
food, portions = "Bread rolls", 30
print(f"{food}: {portions} portions")
```

- `int("30")` turns text into a number; `str(30)` turns a number into text.
- An **f-string** puts values inside text.

Errors are messages

```
TypeError: can only concatenate str (not "int") to str
```

- Read the **last line** first: it names the problem.
- In a notebook, the order you ran cells matters. When confused: **Restart and run all**.

⚖️ Good or risky?

Version A (from an AI assistant)

```
x = 30
y = 4
print(x / y)
```

Version B

```
portions = 30
volunteers = 4
each = portions // volunteers
left = portions % volunteers
print(f"{each} portions each, {left} left over")
```

Principle 1 · Clear names and units

- Code is read by people far more often than it is written.
- A name should say **what** a value means and **in what unit**.
- `hours_left = 5` is clear. `time = 5` is a guess: minutes? hours? a clock time?

> When you review AI code, ask first: *could a coordinator read this?*

You build: the offer summary

```
offer_id = "F001"
food = "Bread rolls"
portions = 30
hours_left = 5
summary = f"{offer_id} · {food} · {portions} portions · {hours_left} hours left"
```

- The coordinator reads this line on a phone, in a hurry.
- The notebook checks four promises: ID, food, portions **with unit**, hours **with unit**.

🧭 Your decision · name your system

```
MY_SETTINGS = {"SYSTEM_NAME": "Battambang Food Share"}
my_system = fr.System(MY_SETTINGS)
```

Decision log	
My choice	…
An alternative	…
Why	…
Who is affected	…

This week's tasks

- ⭐ **Core:** Session 2 practice P1–P4, the offer summary with all ✅, decision log.
- ⭐⭐ **Stretch:** minutes → hours; a summary for a 30-character screen.
- 🔎 **Research 1 (5%):** a real information system — who uses it, what data, one rule, one risk. Share it in the Week 2 warm-up.
- Save your notebook copy in Drive.

Next week · Rules and decisions

Should an offer **wait**, go to **one volunteer**, or need a **team**? You will write the rules — and choose the numbers yourself.